udp UNIVERSIDAD DIEGO PORTALES

Reddit Tech Stack

Distributed Systems

Felipe Maturana
felipe.maturana1@mail.udp.cl

August 23, 2026

What is Reddit?

- Social news aggregation, content rating and discussion platform
- Founded in 2005, now one of the **most visited websites** in the world
- Users submit links, text posts, images and videos to **communities (subreddits)**
- Other users vote content up or down and engage in threaded discussions
- More than **100,000 active communities** covering virtually every topic

Source: [4]

Key concept

Reddit is essentially a **massive distributed voting and conversation engine**, requiring extreme reliability and scale.

Reddit at Scale

Users & Traffic

- **116M** Daily Active Users (DAUs, +19.3% YoY)
- **443.8M** Weekly Active Users (WAUs)
- **2.2B+** visits / month (78% mobile)
- **56.7%** browse logged-out (anonymous)

Content & Engagement

- **3.14B** interactions & comments / year
- **1.4B** native video views / month
- **70M+** on-platform searches / week
- **100k+** subreddits (500+ >1M subscribers)

Challenge

Serving this volume requires a backend handling **massive concurrent reads and writes** with low latency — a distributed systems problem.

Transparency problem

From the user perspective, Reddit *looks like a single system* (**distribution transparency**), but achieving this across thousands of machines poses major challenges.

Why a Distributed System?

A distributed system pursues three core goals — Reddit must satisfy all three simultaneously:

Fault Tolerance

- No single point of failure
- Data replicated across nodes
- Services restart automatically

Scalability

- Horizontal scaling of services
- Traffic spikes (AMAs, events)
- Independent service deployment

Efficiency

- Caching to avoid DB hits
- Async processing of heavy tasks
- Go microservices for low latency

Transparency as a double-edged sword

Distribution **transparency** (access, location, replication, failure) is the goal, but it conflicts with consistency and performance. Reddit must choose *which* transparency properties to

Reddit Core Architecture

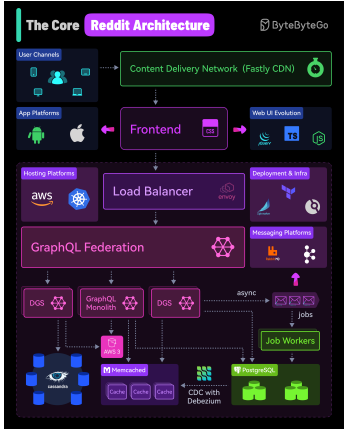


Figure: Reddit's core backend architecture [1]

Architecture: Solving the Three Goals

- GraphQL Gateway — single entry point (access transparency); routes requests to Go microservices
- Go Microservices — each service scales independently (scalability); failures are isolated (fault tolerance)
- PostgreSQL (ThingDB) — primary persistent store with read replicas (replication transparency)
- Memcached & Redis — multi-layer caching (efficiency); absorbs $>$95% of read traffic before it hits the DB
- Kafka + Debezium (CDC) — event streaming for eventual consistency; decouples producers from consumers
- RabbitMQ — async background jobs (votes, emails); improves write throughput
- Cassandra — high-availability NoSQL for time-series and voting data (fault tolerance across DCs)
- AWS / Kubernetes — container orchestration and auto-scaling (scalability)

Source: [1]

Languages & API Layer

Python — Legacy Monolith (r2)

- Original language (2005)
- Handles routing and some business logic
- Bottleneck at scale → being phased out

Go — Modern Microservices

- Rewritten critical paths (comments, feeds)
- Goroutines: lightweight concurrency
- Significantly lower API latency

Source: [4], [5]

GraphQL Federation

- Single unified API gateway for all clients (web, iOS, Android)
- Backend split into **Domain Graph Services (DGS)** subgraphs
- Teams deploy subgraphs independently → **location transparency** for the client
- Prevents tight coupling between frontend and specific backend services

Databases & Caching

PostgreSQL — ThingDB pattern

- EAV (Entity-Attribute-Value) schema
- Stores posts, comments, accounts, votes
- Read replicas for **fault tolerance**
- Schema-less flexibility without migrations

Apache Cassandra

- Distributed, multi-datacenter NoSQL
- Used for vote counts and time-series data
- Tunable consistency model (CAP

Memcached

- First caching layer in front of Postgres
- Caches rendered posts, session tokens
- Absorbs the majority of read traffic

Redis

- Structured cache + pub/sub broker
- Stores leaderboards, rate limits, locks
- Acts as CDC event store (Change Data Capture)

Async Messaging & Infrastructure

RabbitMQ — Background Jobs

- AMQP queue for fire-and-forget tasks
- Votes, notifications, cleanups
- Decouples HTTP request cycle from heavy work

Apache Kafka + Debezium (CDC)

- Debezium tails PostgreSQL WAL logs
- Publishes change events to Kafka topics
- Consumers: search index, spam detection, analytics
- Enables **eventual consistency** without

AWS & Kubernetes (EKS)

- Multi-region AWS deployment
- Kubernetes auto-scales pods on traffic spikes
- **Failure transparency**: crashed pods restart automatically
- “Sister Datastore” migration pattern: parallel writes to old & new DB, compare results, then cut over safely

Bibliography I

[1] ByteByteGo. *Reddit's Core Architecture*. 2024. URL: https://bytebytego.com/guides/reddits-core-architecture/ (visited on 08/23/2026).

[2] Cole Furrh. *Reddit Statistics 2026: Traffic, Users, Ads and Revenue*. 2026. URL: https://www.interteammarketing.com/blog/reddit-statistics-2026 (visited on 08/23/2026).

[3] Himalayas. *Reddit Tech Stack and Tools*. 2024. URL: https://himalayas.app/companies/reddit/tech-stack (visited on 08/23/2026).

[4] StackShare. *Reddit Tech Stack*. 2024. URL: https://stackshare.io/reddit/reddit (visited on 08/23/2026).